

LifeRouter User Manual

Kentalives

June 2026

Contents

1	User Manual	1
1.1	Prerequisites	1
1.2	Installation and Configuration	1
1.2.1	Environment Preparation	1
1.2.2	Build	2
1.2.3	Running as a Service	2
1.2.4	Configuration File	2
1.2.5	Main Fields	3
1.3	Complete Configuration Reference	4
1.3.1	<code>app</code>	4
1.3.2	<code>grid</code>	5
1.3.3	<code>grid.portals</code>	5
1.3.4	<code>pathfinding</code>	5
1.3.5	<code>pathfinding.emergency</code>	6
1.3.6	<code>pathfinding.agent</code>	6
1.3.7	<code>pathfinding.geotrueth</code>	6
1.4	Using the Service From Go	7
1.4.1	Embedded Execution	7
1.4.2	Agent Pathfinding	8
1.4.3	Subscribing to Route State	9
1.4.4	Emergency Routing	9
1.4.5	Rasterized Map Tuning	10
1.5	Common Workflows	10
1.5.1	Validate a Map Before Running It	10
1.5.2	Start an Agent Route	11

1.5.3	Control Agent Movement	11
1.5.4	Observe an Agent Path	12
1.5.5	Run an Emergency	12
1.5.6	Use Multiple Floors	13
1.6	NATS API	13
1.6.1	Agent Subjects	14
1.6.2	Agent Communicator Subjects	14
1.6.3	Emergency Subjects	15
1.6.4	Snapshot Messages in NATS Streams	16
1.7	Domain Types	17
1.8	Common Error Messages	17
1.8.1	Troubleshooting Guide	18

Chapter 1

User Manual

This manual contains the information needed to use the public API of LifeRouter. It is intended for developers who want to integrate the service from Go or from another process that can communicate through NATS. It does not describe the internal algorithmic details; those belong to the design documentation and the technical manual.

1.1 Prerequisites

Using the service requires an accessible NATS process, a rasterizable map configured as a grid, and external geotruth services that provide object information. Public API coordinates are expressed in meters as `[x, y, z]` triples. Conversion between meters and cells depends on `grid.cellsize`; by default, one cell represents 0.2 meters.

Agent identifiers received by the API must exist in geotruth. The service reads the agent's real position, computes a route on the grid, and publishes movement updates through the configured publishing dependency.

1.2 Installation and Configuration

This section describes how to start the service as a standalone process. Embedded execution from another Go program is described later in the API usage section.

1.2.1 Environment Preparation

The project is a Go module and requires a Go version compatible with the version declared in `go.mod`. Besides the service code, the installation needs a reachable NATS server, geotruth services that store and publish object positions, and the map images referenced by the configuration.

Before starting the service, check these points:

- the NATS process is available at the configured URL;

- the map image paths exist from the execution directory;
- grid dimensions match the map that will be rasterized;
- agent identifiers and signaling nodes exist in geotruth;
- floor and portal configuration represents the real space.

1.2.2 Build

The normal build is run from the repository root:

```
make build
```

This target runs `go build` with the default `all` build tag. The Makefile keeps `MODULES` as an optional build-tag override for specific runs, for example:

```
make build MODULES="tag1 tag2"
```

Ordinary builds should use `make build` without overrides.

When Go dependencies change, update module metadata and the `vendor` directory:

```
make deps
```

1.2.3 Running as a Service

The executable loads configuration with `config.LoadConfig(nil)`, prepares default dependencies, initializes the rasterized world, opens the service NATS connection, and registers the dispatcher subjects. The default dependencies also create internal query and publish pools toward geotruth. The process keeps running until it receives a shutdown signal such as `SIGINT` or `SIGTERM`. At that point it closes active agent sessions, stops the emergency if one is running, drains the service NATS connection, and releases closeable dependency resources.

After building, run the generated binary from a directory where `config/config.yaml` and the configured map paths are accessible:

```
./LifeRouter
```

The standalone entry point reuses `embedded.DefaultDependencies`. That function creates separate internal NATS pools for geotruth queries and publishes, and creates a simulated external system for part of the domain data. For a real deployment, the integration should provide its own dependencies through embedded execution or through a service that wraps this module.

1.2.4 Configuration File

The standalone service reads `config/config.yaml`. Omitted fields use `DefaultConfig` values. Embedded calls, tools, or benchmarks can call `LoadConfig` with an explicit path

to use another YAML file, for example a scenario configuration under `data/`. Keys can also be overridden with environment variables that start with `PTHFSERVICE`; dots in key paths become underscores. For example:

```
PTHFSERVICE_APP_SERVERURL=nats://localhost:4222
PTHFSERVICE_GRID_CELLSIZE=0.2
```

The following example shows the main part of a configuration with multiple floors, rasterizable images, and portals:

```
app:
  serverurl: "nats://localhost:4222"
  stderr: true

grid:
  cellsize: 0.2
  rows: 63
  cols: 118
  pxpercell: 10
  wallauracells: 1
  layers:
    - name: "0"
      img: "./data/map.png"
    - name: "1"
      img: "./data/floor1.png"
  portals:
    - from: [1.2, 3.2]
      fromlayer: 0
      to: [1.2, 3.2]
      tolayer: 1
      traversalcost: 15
      bidirectional: true

pathfinding:
  freemoveheight: 2
  agent:
    debug: false
    findpathhandlerworkers: 8
    blockingwaithandlerworkers: 1024
    movefmetershandlerworkers: 16
  geotruth:
    queryconnections: 8
    publishconnections: 8
  emergency:
    debug: false
```

1.2.5 Main Fields

The `app` group defines NATS connectivity and log output. The `serverurl` key contains the NATS server URL. The `stderr`, `fileout`, and `logfile` keys control whether logs are

written to standard error, to a file, or to both.

The **grid** group defines the rasterized representation of the space. **rows** and **cols** are expressed in cells. **pxpercell** indicates how many image pixels correspond to one grid cell. **cellsize** indicates how many meters one cell represents. Entries in **layers** bind each floor to an image, and entries in **portals** connect coordinates on different floors. Portal coordinates are expressed in meters, not cells; portal layer fields are integer indexes into the **layers** list.

The **pathfinding** group contains routing parameters:

- **freemoveheight**: free height used for direct movement and floor queries;
- diagnostic and concurrency parameters: debug traces and worker limits for some agent NATS handlers;
- emergency parameters: lighting hysteresis and preferred-route width;
- agent parameters: vision radius and number of cells between real-position updates;
- geotrueth parameters: number of internal NATS connections used for queries and publishes toward the external service.

Tune these values together with cell size and map scale.

1.3 Complete Configuration Reference

This reference summarizes the fields accepted by the YAML configuration file, either **config/config.yaml** or an explicit path. If a field is omitted, the service uses the value from **DefaultConfig**. Environment variables use the **PTHFSERVICE** prefix and replace dots with underscores. For example, **grid.cellsize** can be overridden with **PTHFSERVICE_GRID_CELLSIZE**.

1.3.1 app

serverurl Text containing the NATS server URL. The default is the local NATS default URL.

stderr Boolean. If **true**, logs are written to standard error. The default is **true**.

fileout Boolean. If **true**, logs are written to a file. The default is **false**.

logfile Text containing the log file path when **fileout** is enabled. The default is an empty string.

1.3.2 grid

layers List of floors. Each element has **name**, the external floor or region identifier, and **img**, the path of the image to rasterize. By default there is one floor named 0 with image `../../data/map.png`.

rows Unsigned integer with the number of grid rows. The default is 61.

cols Unsigned integer with the number of grid columns. The default is 114.

pxpercell Decimal number of image pixels that correspond to one cell. The default is 10.

cellsize Decimal number of meters represented by one cell. It must be greater than 0. The default is 0.2.

wallauracells Integer number of margin cells around walls. It must be greater than or equal to 0. The default is 1.

1.3.3 grid.portals

Each portal connects two coordinates on different floors. Coordinates are expressed in meters as `[x, y]` lists. The service converts them to cells using `grid.cellsize`.

from List `[x, y]` with the source coordinate in meters.

fromlayer Integer index of the source layer in `grid.layers`.

to List `[x, y]` with the destination coordinate in meters.

tolayer Integer index of the destination layer in `grid.layers`.

traversalcost Integer traversal cost for crossing the portal.

bidirectional Boolean. If **true**, the portal can be used in both directions. If **false**, it is used only from source to destination.

No portal is defined by default.

1.3.4 pathfinding

freemoveheight Decimal free height, in meters, used for direct movement and floor queries. It must be greater than 0. The default is 2.

1.3.5 pathfinding.emergency

lighthystheresis Integer cost hysteresis applied to emergency signaling. It must be greater than 0. The default is 100.

priopathwidth Decimal width, in grid cells, of emergency preferred routes. It must be greater than 0. The default is 1.

debug Boolean. If **true**, enables diagnostic output for emergency pathfinding. The default is **false**.

1.3.6 pathfinding.agent

visionradiuscells Unsigned integer with the agent vision radius in cells. It must be greater than 0. The default is 10.

cellsforrealupdate Integer number of cells the agent can move before updating its real position. It must be greater than 0. The default is 5.

maxreusableworldlinkedfloors Unsigned integer with the maximum number of linked floors that can be reused in an agent world. The default is 2.

sparsevaluepagedirectory Boolean. Enables sparse storage for the agent value directory. The default is **false**.

findpathhandlerworkers Integer maximum number of worker goroutines that can handle **AgentFindPath** NATS requests in parallel. The default is **GOMAXPROCS**.

blockingwaithandlerworkers Integer maximum number of worker goroutines that can handle **BlockingWait** requests in parallel. The default is 1024.

movefmetershandlerworkers Integer maximum number of worker goroutines that can handle **MoveFMeters** requests in parallel. The default is $2 * \text{GOMAXPROCS}$.

debug Boolean. If **true**, enables diagnostic output for agent pathfinding. The default is **false**.

These worker limits affect only the NATS handler layer. They do not change the pathfinding algorithm and do not parallelize the steps of a single agent. Orders for one communicator remain ordered by its internal protocol, while requests for different agents can be handled concurrently.

1.3.7 pathfinding.geotruth

queryconnections Integer number of NATS connections used internally for geotruth queries such as object data, local vision, and floor resolution. The default is **GOMAXPROCS**.

publishconnections Integer number of NATS connections used internally to publish position updates to geotruth. The default is **GOMAXPROCS**.

These values do not change the NATS connection used by a public client to call the `pathfinding` API. They only affect internal traffic from the pathfinding service to `geotruth`. Increasing them can reduce contention when many agents query local vision and publish positions, but it also increases the number of connections and goroutines held by the process. Values lower than 1 are treated as one connection.

1.4 Using the Service From Go

The recommended public surface for Go integrations is formed by these packages:

- `embedded`
- `pkg/pathfinding`
- `pkg/emergency`
- `pkg/domain`
- `pkg/config`
- `pkg/subjects`
- `pkg/tuning`

1.4.1 Embedded Execution

The `embedded` package starts the same dispatcher used by the executable, but inside the caller process. This option is useful in integrations, tests, and tools that want explicit control over the service lifecycle.

```
ctx := context.Background()
cfg := embedded.DefaultConfig()

dep, err := embedded.DefaultDependencies(ctx, cfg)
if err != nil {
    return err
}

dispatcher, err := embedded.Run(ctx, cfg, dep)
if err != nil {
    return err
}
defer dispatcher.Shutdown()
```

`DefaultConfig` returns convenient local values. In a real integration, review at least the NATS URL, map image paths, grid dimensions, portals, and external dependencies. `DefaultDependencies` creates internal NATS pools for `geotruth` query and publishing, and uses a simulated external system. In a real deployment, provide implementations of `domain.ExternalSystem`, `domain.GeoQuery`, and `domain.GeoPublish`. The `GeoPublish`

implementation publishes position updates and returns a commit acknowledgement; the integrator remains responsible for releasing any external clients or connections created outside the default pools.

To validate a local installation or integration, run:

```
make test
```

This command runs the full suite and generates `coverage.out`, a local per-package coverage report. To measure code exercised by all tests in the module, including black-box and integration tests that execute other packages, run `make test-cover-full`. The corresponding profile is `coverage-full.out` and can be opened with `make cover-full-html`.

1.4.2 Agent Pathfinding

The `pkg/pathfinding` package provides a client over an existing NATS connection. The client does not own the connection; the caller remains responsible for closing it.

```
pf := pathfinding.New(nc)
goal := [3]float64{10.0, 4.0, 0.0}

comm, err := pf.AgentFindPath(ctx, goal, "agent-1", 2.0, 0)
if err != nil {
    return err
}

if err := comm.MoveNCells(ctx, 5); err != nil {
    return err
}

done, err := comm.BlockingWait()
if err != nil {
    return err
}
<-done

if err := comm.ExitError(ctx); err != nil {
    return err
}
```

`AgentFindPath` starts adaptive routing toward the given goal. `defaultCellsPerSecondMovement` defines automatic speed in cells per second. A zero value leaves the session stopped until explicit movement commands arrive. `moveNSteps` requests an initial number of cells to move.

To estimate the cost without moving the agent, use `AgentNaivePathCost`:

```
cost, err := pf.AgentNaivePathCost(ctx, goal, "agent-1")
```

An active session returns an `AgentCommunicator`. It can change speed, pause, terminate, move by cells, move by meters, check whether the run is still moving, and query the

terminal error. The terminal result remains available for a short interval after completion; after that time, or after another session for the same agent starts, it may no longer be available.

1.4.3 Subscribing to Route State

`AgentPathSub` observes route snapshots for an agent. The channel returns a map by floor: the key is the floor name and the value is a list of cell states encoded with `domain.CellState`.

```
states, err := pf.AgentPathSub(ctx, "agent-1")
if err != nil {
    return err
}

for snapshot := range states {
    _ = snapshot["0"]
}
```

The channel closes when the service reports that the subscription has ended. On large maps, the service may transport one snapshot as several compressed NATS messages; `AgentPathSub` assembles those chunks before returning the map on the Go channel.

1.4.4 Emergency Routing

The `pkg/emergency` package controls the emergency flow field. `Start` receives a list of exits, expressed as meter coordinates, and optionally per-floor preference graphs.

```
em := emergency.New(nc)
goals := [] [3]float64{
    {1.0, 1.0, 0.0},
}

if err := em.Start(ctx, goals, nil, 2.0); err != nil {
    return err
}
defer em.Stop(ctx)
```

Flow snapshots can be observed with `FlowSub`. Each cell returns a direction encoded with `domain.Direction`, including planar directions, diagonal directions, and transitions between floors.

```
flows, err := em.FlowSub(ctx)
if err != nil {
    return err
}

for snapshot := range flows {
    _ = snapshot["0"]
}
```

As with agent route state, `FlowSub` hides the chunked transport used when the complete flow field is too large for one NATS message.

1.4.5 Rasterized Map Tuning

The `pkg/tuning` package helps inspect the grid generated from an image. It does not run pathfinding, but it lets users check dimensions, scale, and wall aura before using an image as a service map. Rasterization uses the same production path as the service: traversable zones must be encoded in the red channel, not only as visually light or grayscale areas.

```
grid, err := tuning.TraversableImageRender(
    "./data/map.png",
    63,
    118,
    10,
    1,
)
if err != nil {
    return err
}

fmt.Println(grid)
```

For large grids, the same structure can open a temporary web visualization in the system browser:

```
if err := grid.OpenVisualizer(); err != nil {
    return err
}
```

1.5 Common Workflows

1.5.1 Validate a Map Before Running It

Before starting the service with a new image, rasterize it with the same logic used by the service. This detects incorrect routes, wrong dimensions, walls too close to traversable space, or maps whose traversability information is not in the red channel.

```
grid, err := tuning.TraversableImageRender(
    "./data/map.png",
    rows,
    cols,
    pxPerCell,
    wallAuraCells,
)
if err != nil {
    return err
}
```

```
fmt.Println(grid)
```

If the printed grid does not represent the space well, review `rows`, `cols`, `pxpercell`, `cellsize`, and `wallauracells`. Also confirm that traversable pixels have enough red channel value to pass the internal rasterization threshold before starting route calculations.

1.5.2 Start an Agent Route

The basic flow requires the service to be active, NATS to be available, and the agent to exist in `geotruth`. The initial position is read from `geotruth` and the goal is supplied as a meter coordinate.

```
pf := pathfinding.New(nc)
goal := [3]float64{x, y, z}

comm, err := pf.AgentFindPath(ctx, goal, agentID, 2.0, 0)
if err != nil {
    return err
}
```

After starting the session, wait for completion and query the final result:

```
done, err := comm.BlockingWait()
if err != nil {
    return err
}
<-done

if err := comm.ExitError(ctx); err != nil {
    return err
}
```

If `ExitError` returns `nil`, the session ended without a domain error. Otherwise, interpret the error using the error-message section.

1.5.3 Control Agent Movement

When the initial speed is zero, the session is prepared but does not advance until it receives an explicit movement command. This mode is useful for step-by-step simulations or for synchronizing movement with another system.

```
comm, err := pf.AgentFindPath(ctx, goal, agentID, 0, 0)
if err != nil {
    return err
}

if err := comm.MoveNCells(ctx, 5); err != nil {
    return err
}
```

Movement can also be requested in meters. The call blocks until the distance is consumed or the session ends:

```
remaining, err := comm.MoveFMeters(ctx, 1.5)
if err != nil {
    return err
}
_ = remaining
```

To switch from manual mode to continuous movement, update the speed. A zero value pauses automatic movement without destroying the session:

```
_ = comm.UpdateMovementSpeed(ctx, 3.0)
_ = comm.Stop(ctx)
_ = comm.Terminate(ctx)
```

1.5.4 Observe an Agent Path

Path snapshots can be used to visualize the search state. Start the session, subscribe to the agent identifier, and process updates until the channel closes.

```
updates, err := pf.AgentPathSub(ctx, agentID)
if err != nil {
    return err
}

for snapshot := range updates {
    cellsByFloor := snapshot
    _ = cellsByFloor
}
```

Each snapshot is a map by floor. Cell values are interpreted using `domain.CellState`. For large maps, the Go API still yields one complete snapshot per iteration; the split into NATS messages matters only for direct NATS consumers.

1.5.5 Run an Emergency

The emergency flow starts by defining one or more exits in meters. Optionally, preference graphs can be supplied to favor specific routes per floor.

```
em := emergency.New(nc)
goals := [][]float64{
    {exitX, exitY, exitZ},
}

if err := em.Start(ctx, goals, routeGraphs, 2.0); err != nil {
    return err
}
```

While the emergency is active, subscribe to the flow field. The channel closes when the service reports that publishing has ended:

```
flows, err := em.FlowSub(ctx)
if err != nil {
    return err
}

for snapshot := range flows {
    directionsByFloor := snapshot
    _ = directionsByFloor
}
```

The Go channel yields complete per-floor maps. Transport chunking for large fields is handled by the `pkg/emergency` client.

When the emergency situation ends, call `Stop` to stop computation and remove preferred routes from the grid. The operation is idempotent: if there is no active emergency, or if another call already started the stop, it returns success once the system is stopped.

```
if err := em.Stop(ctx); err != nil {
    return err
}
```

1.5.6 Use Multiple Floors

The previous flows also work in multi-floor spaces if the configuration includes the corresponding layers and portals. Public coordinates are still expressed as `[x, y, z]`; the `z` value relates the real position to geotruuth regions and grid floors.

For this case, verify that every floor has an image, portals connect traversable cells, and the external system knows the region heights. If any of these pieces is missing, a route that appears valid on one floor may fail when crossing to another.

1.6 NATS API

Integrations that do not use the Go wrappers can call NATS subjects directly. Request/reply calls return a JSON wrapper from the shared `messages` package:

```
{
  "ok": true,
  "error": "",
  "error_code": "",
  "data": ...
}
```

When `ok` is `false`, `error` contains the remote message and `error_code` can contain a stable code for known cross-process errors. Go clients should prefer `messages.Data[T]`

and `messages.Err` to decode responses. Local wrapper errors, such as the public operation or NATS request/response phase, are not transmitted in this JSON; they appear only in the error returned by the Go client.

Go consumers that use NATS directly can import subject constants from `pkg/subjects`. Consumers in other languages can use the literal subject names shown below.

1.6.1 Agent Subjects

Subject	Input	Output
<code>pathfinding.agent_find_path</code>	<code>AgentFindPathParams</code>	communicator id
<code>pathfinding.agent_naive_path_cost</code>	<code>AgentNaivePathCostParams</code>	cost
<code>pathfinding.agent_path_watch</code>	agent identifier	stream on reply subject

Table 1.1: NATS subjects for starting and observing agent pathfinding.

`AgentFindPathParams` has this shape:

```
{
  "goal": [10.0, 4.0, 0.0],
  "agentid": "agent-1",
  "defaultcellsperssecondmovement": 2.0,
  "movensteps": 0
}
```

`AgentNaivePathCostParams` uses `goal` and `agentid`. The path subscription receives the agent identifier:

```
{"id": "..."}

```

The path subscription sends small messages to the reply subject:

```
{
  "done": false,
  "cells": {
    "0": [0, 1, 4, 5]
  }
}
```

1.6.2 Agent Communicator Subjects

Operations on an active agent use the identifier returned by:

```
pathfinding.agent_find_path

```

In the current implementation, this identifier is the agent identifier.

Subject	Input	Output
agent_comm. update_movement_speed	id and new speed	OK
agent_comm. blocking_wait	id	notification on reply subject
agent_comm. is_moving	id	boolean
agent_comm. exit_error	id	OK or final error
agent_comm. terminate	id	OK
agent_comm. stop	id	OK
agent_comm. move_n_cells	id and cell count	OK
agent_comm. move_f_meters	id and distance in meters	remaining meters

Table 1.2: NATS subjects for controlling an agent session.

The JSON bodies for these control subjects have these shapes:

```
{ "id": "..."}
{ "id": "...", "float": 2.0}
{ "id": "...", "n": 5}
```

The `float` field represents the new speed in `update_movement_speed` and the distance in meters in `move_f_meters`.

`move_f_meters` blocks until the requested distance is consumed or the session ends. If the agent stopped before completing the distance, the response can indicate `ErrAgentExitedWithMetersLeft`.

1.6.3 Emergency Subjects

Subject	Input	Output
emergency.start	<code>EmergencyPathParams</code>	OK
emergency.stop	no body	OK, also when no emergency was active
emergency. flow_watch	no body	stream on reply subject

Table 1.3: NATS subjects for controlling emergency routing.

`EmergencyPathParams` includes exits, preference graphs, and update frequency:

```
{
  "goals": [[1.0, 1.0, 0.0]],
  "preferencegraph": [],
  "updatetickspersecond": 2.0
}
```

`emergency.flow_watch` messages use the same close criterion as agent path streams. For small fields, the format is:

```
{
  "done": false,
  "cells": {
    "0": [1, 3, 10]
  }
}
```

1.6.4 Snapshot Messages in NATS Streams

`pathfinding.agent_path_watch` and `emergency.flow_watch` share the same snapshot stream format. Go wrapper users do not need to handle this format directly, but direct NATS consumers in other languages must accept two valid message shapes. The first shape is the small-message envelope preserved from the original contract:

```
{
  "done": false,
  "cells": {
    "0": [0, 1, 4, 5]
  }
}
```

The second shape is used when the full snapshot is too large for one NATS message. Each message carries one compressed chunk:

```
{
  "encoding": "gzip+json",
  "snapshotId": "42",
  "chunkIndex": 0,
  "chunkCount": 3,
  "data": "..."
}
```

Consumers must group chunks by `snapshotId`, order them by `chunkIndex`, base64-decode each `data` field, concatenate the resulting bytes, decompress the result with `gzip`, and decode the JSON as a map by floor. The stream ends when this message arrives:

```
{"done": true}
```

This chunking avoids requiring a larger NATS maximum payload. Default NATS configurations commonly limit one message to about 1 MiB.

1.7 Domain Types

Costs use `domain.Cost`. `COST_EMPTY_SPACE` represents normal movement cost and `COST_UNREACHABLE` represents an unreachable cell.

Route visualization states use `domain.CellState`:

- `STATE_Unvisited`
- `STATE_Queued`
- `STATE_Expanded`
- `STATE_Inconsistent`
- `STATE_OnPath`
- `STATE_Obstacle`

Emergency directions use `domain.Direction`. In addition to the eight planar directions, `DIR_IN` and `DIR_OUT` represent transitions between floors, and `DIR_UNKNOWN` means no direction is defined.

Emergency preference graphs use `domain.RouteGraph`. Each graph contains a floor index, a list of `GraphWaypoint`, and a list of `GraphEdge`. Waypoints use real-world meter coordinates. Graphs do not replace the rasterized map; they modify costs to favor specific routes. If an edge omits `Weight`, the service treats it as weight 1.

1.8 Common Error Messages

Error	Meaning for the user
<code>ErrAgentCommNotFound</code>	There is no active or recently completed session with the given identifier.
<code>ErrAgentNoPath</code>	The service could not find a possible route to the goal.
<code>ErrAgentStillRunning</code>	The final error was queried while the session was still running.
<code>ErrAgentTerminated</code>	The session was explicitly terminated.
<code>ErrAgentExitedWithMetersLeft</code>	The meter movement command could not be completed because the session ended first.
<code>ErrEmergencyStillRunning</code>	An emergency was started while another emergency was already active.
<code>ErrDispatcherShuttingDown</code>	The service is shutting down and no longer accepts new sessions.

Table 1.4: Main public API errors.

Besides these domain errors, a response can fail because of NATS connectivity, invalid JSON, coordinates outside the grid, or unavailable external geotruth services. In those cases, review the error message together with service configuration and dependency health.

The public Go wrappers combine errors so callers can classify the operation without depending on message text. One error can identify the operation, the NATS phase, and the domain cause. For example, after `Emergency.Start`, a client can check:

```
errors.Is(err, domain.ErrEmergencyStart)
errors.Is(err, domain.ErrNATSResponse)
errors.Is(err, domain.ErrEmergencyStillRunning)
```

The main phase sentinels are `ErrNATSRequest`, `ErrNATSResponse`, `ErrNATSSubscription`, and `ErrNATSPublish`. Operation sentinels identify calls such as `ErrAgentFindPath`, `ErrAgentCommStop`, or `ErrEmergencyFlowSub`.

1.8.1 Troubleshooting Guide

When a call fails or blocks, first review these operational points:

NATS connection. Check that the NATS server is active, that `app.serverurl` matches the real address, and that the client uses the same server. Timeouts usually indicate that the service is not subscribed to the requested subject or that NATS is unreachable.

Objects in geotruth. Verify that the agent identifier exists in geotruth before calling `AgentFindPath`. The service reads the agent's initial position from that system.

Images and execution directory. Confirm that `grid.layers.img` paths exist from the directory where the process is launched. A path that works from the repository may fail from another directory.

Grid dimensions and scale. Review `rows`, `cols`, `pxpercell`, and `cellsize` when the route does not match the real space. Use `pkg/tuning` before deployment to render the grid and detect scale or wall-rasterization problems.

Cell coordinates versus rows and columns. Internal grid coordinates are written as `X,Y`, where `X` is the column and `Y` is the row. Printed output and visualizers are usually read as row-column, so a cell observed as `(row, column)` must be converted to `(X: column, Y: row)` before multiplying by `cellsize`.

Floors and regions. If a coordinate `[x, y, z]` does not resolve to the expected floor, review the external system that translates real points to regions and the names defined in `grid.layers`. Floor names must be consistent between service configuration and geotruth.

Portals between floors. In multi-floor environments, check that each portal joins traversable cells, source and destination layer indexes exist, and the traversal cost is positive. A badly placed portal can make an inter-floor route impossible even if each floor is traversable by itself.

Route not found. For `ErrAgentNoPath`, inspect whether the exit or goal is inside a blocked zone, dynamic obstacles closed the only available connection, or the rasterized grid isolated the initial cell.

Blocking calls. `BlockingWait` and `MoveFMeters` can wait until the session ends or the agent consumes the requested distance. If many simultaneous calls are made for different agents, review `pathfinding.agent.blockingwaiterhandlers` and `pathfinding.agent.movefmetershandlerworkers`. These values only limit NATS handlers; they do not change route calculation or command ordering for a single agent.